

Computergrafik

PD Prof. Dr. rer nat.
Marco Block-Berlitz

Sommersemester 2026



Diskrete Geometrie: Konvexität und Kreisalgorithmen

„Nichts ist so beständig wie der Wandel.“ (Heraklit von Ephesus)

PD Prof. Dr. Marco Block-Berlitz

Entscheidungsvariable nutzen: „innerhalb“

Wir wissen, dass $F(x,y) = 0$ ist, für alle (x,y) , die auf dem Kreis liegen.

Weiterhin gilt:

- $F(x,y) < 0$, falls (x,y) innerhalb des Kreises.
- $F(x,y) \geq 0$, falls (x,y) außerhalb des Kreises.

Aus dieser Beobachtung heraus können wir uns eine Entscheidungsvariable Δ vorstellen:

$\Delta = F(M) = F(x+1, y-0.5)$

$\Delta < 0$, M liegt innerhalb des Kreises

$\Delta \geq 0$, M liegt außerhalb des Kreises

$\Delta \geq 0 \Rightarrow (x+1, y-1)$

Konvexitätstest nach Paul Bourke

Ein Polygon ist **konvex**, wenn für alle Punktepaare in Bezug auf den jeweils nachfolgenden gilt, dass es bei $F(x,y)$ keinen Vorzeichenwechsel gibt [1]. Ansonsten ist das Polygon **konkav**.

konvex

konkav

17

44

Ziele und Inhalt

- Polygone konvex/konkav
- Punkt versus Gerade
- Punkt versus Gerade – Sonderfälle
- Punkt versus Gerade – Allgemeine Form
- Konvexitätstest nach Paul Bourke
- Zeichnen von Linien und Kreisen
- Kreise – parametrisiert
- Kreise – Version 1
- Kreise – Version 2
- Kreise – als Funktion
- Kreise – Version 3
- Kreise – Version 4
- Kreise – nach Bresenham
- Beobachtung der Kreisfunktion
- Entscheidungsvariable „außerhalb“
- Entscheidungsvariable „innerhalb“
- Änderungen unter der Lupe
- Kreise – Bresenhamversion 1
- Kreise – Bresenhamversion 2
- Kreise vollständig zeichnen
- Ellipsen zeichnen
- Kreis zu Ellipse – parametrisiert
- Fragestellungen zum Vorlesungsteil
- Praktische Aufgaben

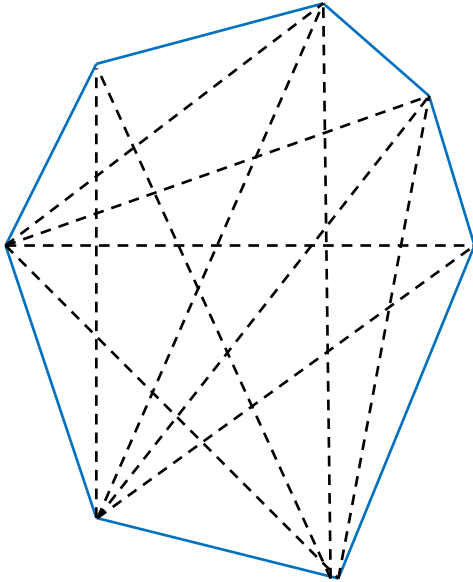




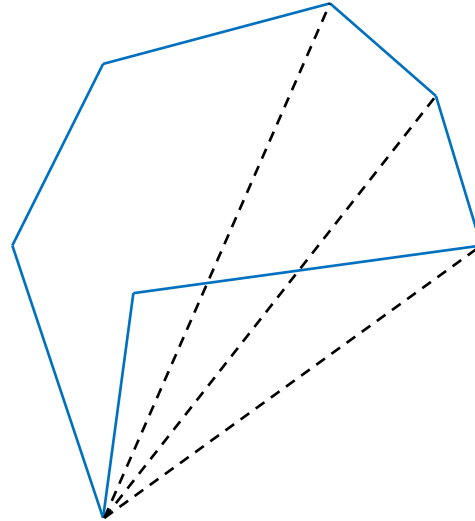
Linien allein sind sicherlich schön,
machen aber noch nicht glücklich ...

Polygone konvex und konkav

Wenn wir Linien verbinden, können wir Polygone erzeugen. Wir unterscheiden zwischen **konvexen** (alle Innenwinkel $\leq 180^\circ$) und **konkaven Polygonen** (mindestens ein Innenwinkel $> 180^\circ$).



konvex



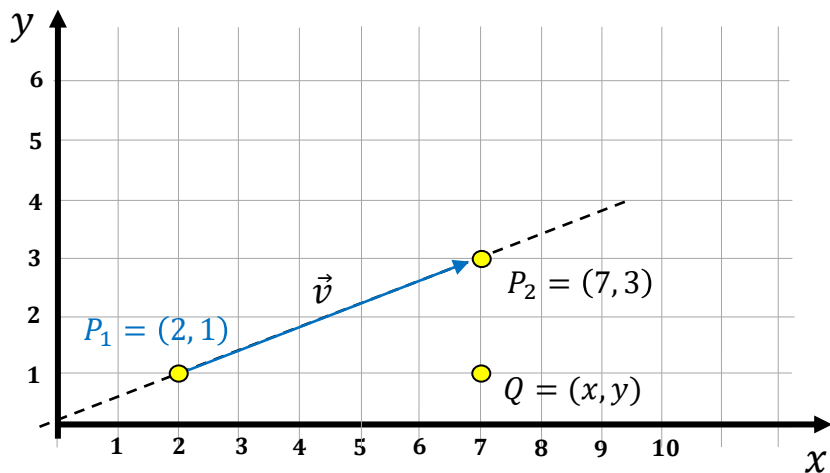
konkav



Wie stelle ich fest, ob es **konvex** oder **konkav** ist?

Punkt versus Gerade

Eine Gerade sei definiert durch die Punkte P_1 und P_2 mit dem Richtungsvektor $\vec{v}$. Ein Punkt $Q(x, y)$ kann relativ zu dieser Gerade jetzt **links**, **rechts** oder **auf** der gegebenen Geraden im gleichen Raum liegen.



$$\vec{u} = \overrightarrow{p_1} + r \cdot \vec{v} = \begin{bmatrix} 2 \\ 1 \end{bmatrix} + r \cdot \begin{bmatrix} 7 - 2 \\ 3 - 1 \end{bmatrix}$$

Punkt versus Gerade

Jetzt wollen wir das **Gleichungssystem lösen** und in eine nützliche Form bringen, über die wir leicht entscheiden können, ob ein Punkt auf der Geraden liegt oder zu welcher Seite:

$$\vec{u} = \begin{bmatrix} 2 \\ 1 \end{bmatrix} + r \cdot \begin{bmatrix} 7 - 2 \\ 3 - 1 \end{bmatrix}$$

$$x = 200 + 500r \quad (1)$$

$$y = 100 + 200r \quad (2)$$

$$x = 2 + 5r \quad Gl(1)$$

$$y = 1 + 2r \quad Gl(2)$$

$$2x = 400 + 1000r \quad (3) = 2 \cdot (1)$$

$$2x = 4 + 10r \quad Gl(3) = 2 \cdot Gl(1)$$

$$-5y = -500 - 1000r \quad (4) = -5 \cdot (2)$$

$$-5y = -5 - 10r \quad Gl(4) = -5 \cdot Gl(2)$$

$$2x - 5y = -100 \quad (5) = (3) + (4)$$

$$2x - 5y = 4 + 10r - 5 - 10r \quad Gl(5) = Gl(3) + Gl(4)$$

$$2x - 5y + 100 = 0$$

$$2x - 5y = -1$$

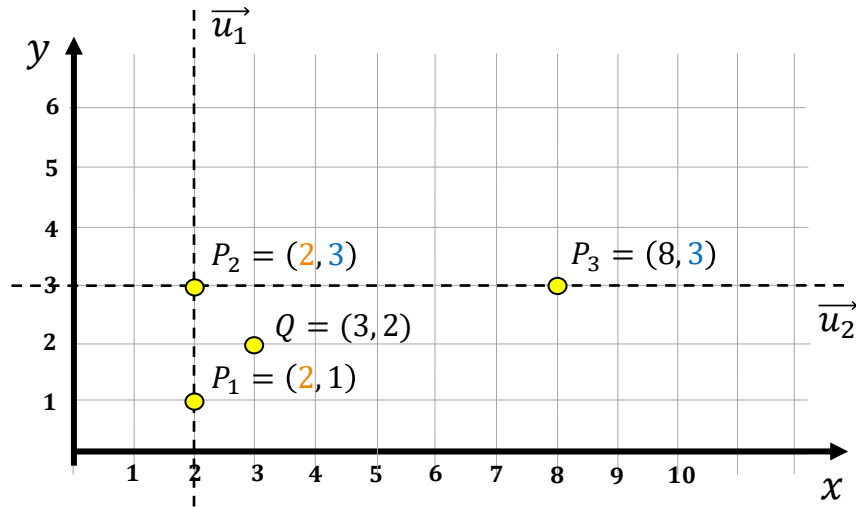
$$2x - 5y + 1 = 0 \quad Gl(5) \text{ umformen}$$

Alle Punkte (p_x, p_y) , die die Gleichung $F(x, y) = 2x - 5y + 1$ mit $F(p_x, p_y) = 0$ erfüllen, liegen **auf** der Geraden.

Alle Punkte mit $F(p_x, p_y) > 0$ liegen **rechts** und $F(p_x, p_y) < 0$ entsprechend **links** von der Geraden.

Punkt versus Gerade – Sonderfälle

Schauen wir uns zusätzlich noch zwei **Sonderfälle** zweier Geraden (mit jeweils gleicher x - bzw. gleicher y -Komponente) an:

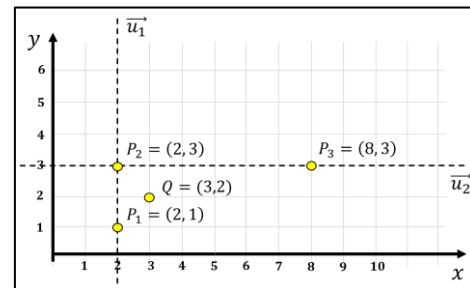


$$\vec{u}_1 = \begin{bmatrix} 2 \\ 1 \end{bmatrix} + r \cdot \begin{bmatrix} 2 - 2 \\ 3 - 1 \end{bmatrix}$$

$$\vec{u}_2 = \begin{bmatrix} 2 \\ 3 \end{bmatrix} + r \cdot \begin{bmatrix} 8 - 2 \\ 3 - 3 \end{bmatrix}$$

Punkt versus Gerade – Sonderfälle

Schauen wir uns zusätzlich noch zwei **Sonderfälle** zweier Geraden (mit jeweils gleicher x - bzw. gleicher y -Komponente) an:



Fall 1)

$$\vec{u}_1 = \vec{p}_1 + r \cdot \vec{v} = \begin{bmatrix} 2 \\ 1 \end{bmatrix} + r \cdot \begin{bmatrix} 2 - 2 \\ 3 - 1 \end{bmatrix}$$

Jetzt die Umformung zur Normalform:

$$x = 2 + 0r \quad Gl(1)$$

$$y = 1 + 2r \quad Gl(2)$$

$$x - 2 = 0 \quad Gl(1) \text{ umformen}$$

Fall 2)

$$\vec{u}_2 = \vec{p}_2 + r \cdot \vec{v} = \begin{bmatrix} 2 \\ 3 \end{bmatrix} + r \cdot \begin{bmatrix} 8 - 2 \\ 3 - 3 \end{bmatrix}$$

Jetzt die Umformung zur Normalform:

$$x = 2 + 6r \quad Gl(1)$$

$$y = 3 + 0r \quad Gl(2)$$

$$y - 3 = 0 \quad Gl(2) \text{ umformen}$$



Glücklicherweise können wir eine **allgemeine Lösung** analytisch herleiten!

Punkt versus Gerade – Allgemeine Form

Allgemein lässt sich die Bestimmung, ob ein gegebener Punkt **auf** einer Geraden oder sich **rechts** bzw. **links** davon befindet wie folgt ableiten:

$$\begin{bmatrix} u_x \\ u_y \end{bmatrix} = \begin{bmatrix} p_x \\ p_y \end{bmatrix} + r \cdot \begin{bmatrix} v_x \\ v_y \end{bmatrix} \quad \longrightarrow \quad \begin{aligned} u_x &= p_x + v_x r & Gl(1) \\ u_y &= p_y + v_y r & Gl(2) \end{aligned}$$

Jetzt lösen wir beide Gleichungen nach r auf und setzen sie gleich:

$$\begin{aligned} (u_x - p_x)/v_x &= (u_y - p_y)/v_y \\ (u_x - p_x) \cdot v_y &= (u_y - p_y) \cdot v_x \\ u_x \cdot v_y - p_x \cdot v_y &= u_y \cdot v_x - p_y \cdot v_x \end{aligned}$$

Nach der Eliminierung von r und der Umformung zur **Normalform**, erhalten wir also:

$$F(u_x, u_y) = u_x \cdot v_y - p_x \cdot v_y - u_y \cdot v_x + p_y \cdot v_x$$

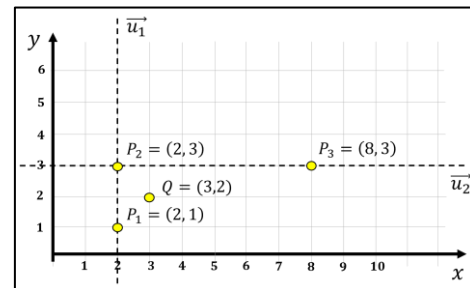
Jetzt können wir durch Umbenennung eine einfache **Entscheidungsfunktion** konstruieren:

$$F(\mathbf{x}, \mathbf{y}) = \mathbf{x} \cdot \mathbf{v}_y - p_x \cdot v_y - \mathbf{y} \cdot \mathbf{v}_x + p_y \cdot v_x$$

Punkt versus Gerade – Sonderfälle in allgemeiner Form

Schauen wir uns die beiden **Sonderfälle** noch einmal mit der allgemeinen Lösung an:

$$F(x, y) = x \cdot v_y - p_x \cdot v_y - y \cdot v_x + p_y \cdot v_x$$



Fall 1)

$$\vec{u}_1 = \vec{p}_1 + r \cdot \vec{v} = \begin{bmatrix} 2 \\ 1 \end{bmatrix} + r \cdot \begin{bmatrix} 2 - 2 \\ 3 - 1 \end{bmatrix}$$

Jetzt die Umformung zur allg. Normalform:

$$\begin{aligned} F(x, y) &= x \cdot 2 - 2 \cdot 2 - y \cdot 0 + 1 \cdot 0 \\ &= 2x - 4 \end{aligned}$$

Fall 2)

$$\vec{u}_2 = \vec{p}_2 + r \cdot \vec{v} = \begin{bmatrix} 2 \\ 3 \end{bmatrix} + r \cdot \begin{bmatrix} 8 - 2 \\ 3 - 3 \end{bmatrix}$$

Jetzt die Umformung zur allg. Normalform:

$$\begin{aligned} F(x, y) &= x \cdot 0 - 2 \cdot 0 - y \cdot 6 + 3 \cdot 6 \\ &= -6y + 18 \end{aligned}$$



Wir haben mit der Normalform eine Entscheidungsfunktion für einen Punkt: **links**, **rechts**, **koliniear**

Zusammenhang von Normalform und Kreuzprodukt

Eine Gerade ist definiert durch Punkt $P = (p_x, p_y)$ mit dem Richtungsvektor $\vec{v} = (v_x, v_y)$ und führt für einen weiteren Punkt $Q = (x, y)$ zur Normalform $F(x, y)$ mit:

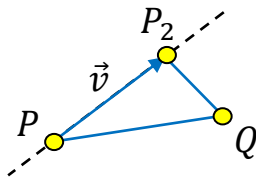
$$F(Q) = F(x, y) = x \cdot v_y - p_x \cdot v_y - y \cdot v_x + p_y \cdot v_x$$

oder umgestellt zu:

$$F(x, y) = v_y(x - p_x) - v_x(y - p_y)$$

Das ist die **Entscheidungsfunktion**, für einen neuen Punkt $Q = (x, y)$

- $F(x, y) > 0 \Rightarrow$ Punkt Q links
- $F(x, y) < 0 \Rightarrow$ Punkt Q rechts
- $F(x, y) = 0 \Rightarrow$ Punkt Q auf der Geraden



Überführung der Normalform in das Kreuzprodukt:

$$F(x, y) = v_y(x - p_x) - v_x(y - p_y)$$

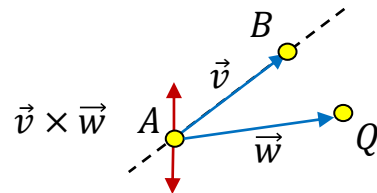
mit $\vec{v} = (v_x, v_y)$ und $\vec{w}$ ist $\vec{w} = (x - p_x, y - p_y)$

Entspricht also:

$$F(x, y) = -(\vec{v} \times \vec{w})$$



$$\vec{v} \times \vec{w} = v_x w_y - v_y w_x$$

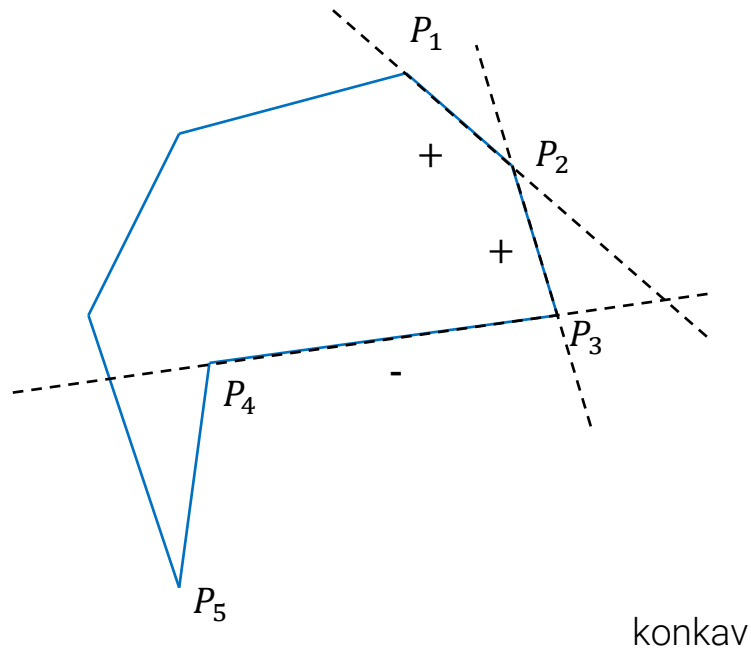
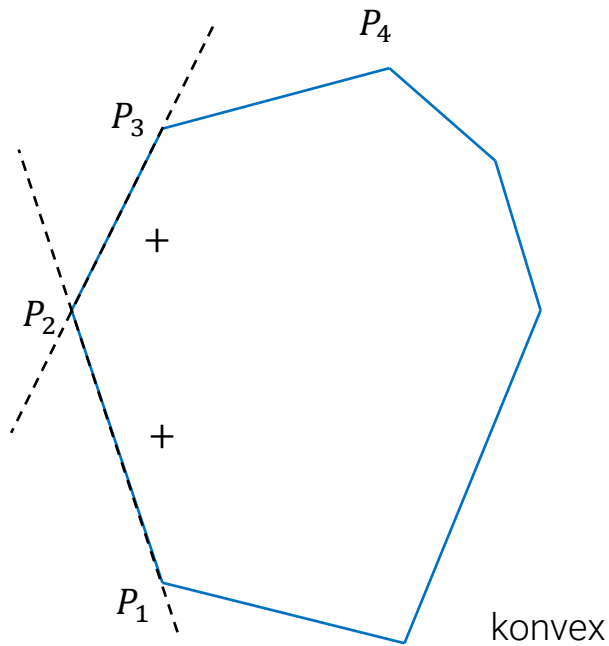




Konvexitätstest für Polygone nach Paul Bourke

Konvexitätstest nach Paul Bourke

Ein Polygon ist **konvex**, wenn für alle Punktpaare in Bezug auf den jeweils nachfolgenden gilt, dass es bei $F(x, y)$ keinen Vorzeichenwechsel gibt [1]. Ansonsten ist das Polygon **konkav**.



[1] <https://paulbourke.net/geometry/polygonmesh/>

Algorithmus und Implementierung von Paul Bourke

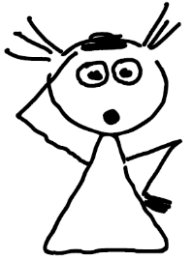
Schauen wir uns die Abarbeitung des Algorithmus für ein gegebenes Polygone ($n > 3$) schrittweise an. Im ersten Beispiel sehen wir ein konvexes und anschließend zwei konkave Polygone.

```
isConvex(P[0], P[1], ..., P[n-1]):  
    sign=0           // aktuelle Richtung (+1/-1)  
    for (i=0 to n-1):  
        z = orientation(P[i],  
                        P[(i+1) mod n],  
                        P[(i+2) mod n])  
  
        if (z==0) continue // kollinear  
        currentSign = sign(z)  
        if (sign==0)  
            sign=currentSign  
        else if (currentSign!=sign)  
            return CONCAVE  
    if (sign==0)  
        return DEGENERATE // alles kollinear  
    return CONVEX
```





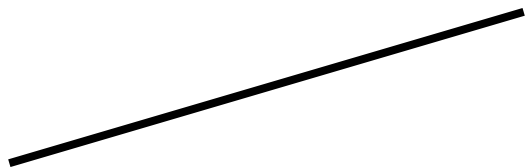
Konvexitätstest funktioniert nur zuverlässig für
einfache Polygone (ohne Selbstüberschneidung).



Jetzt wollen wir zum **Zeichnen von Kreisen** kommen.

Zeichnen von Linien und Kreisen

Schauen wir uns noch einmal den Werdegang des Linienzeichnens an und überlegen uns, was das für einen **Kreis** bedeuten könnte.



Parametrisiert:

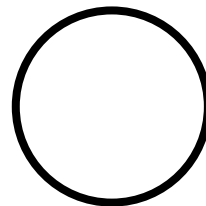
$$x = f_x(t) \quad y = f_y(t)$$

Funktionsgleichung:

$$y = f(x)$$

Bresenham-Linie:

```
x++; if (...) y--; ...
```



Parametrisiert:

$$x = f_x(t) \quad y = f_y(t)$$

Funktionsgleichung (Kreis):

$$y = f(x)$$

Bresenham-Kreis:

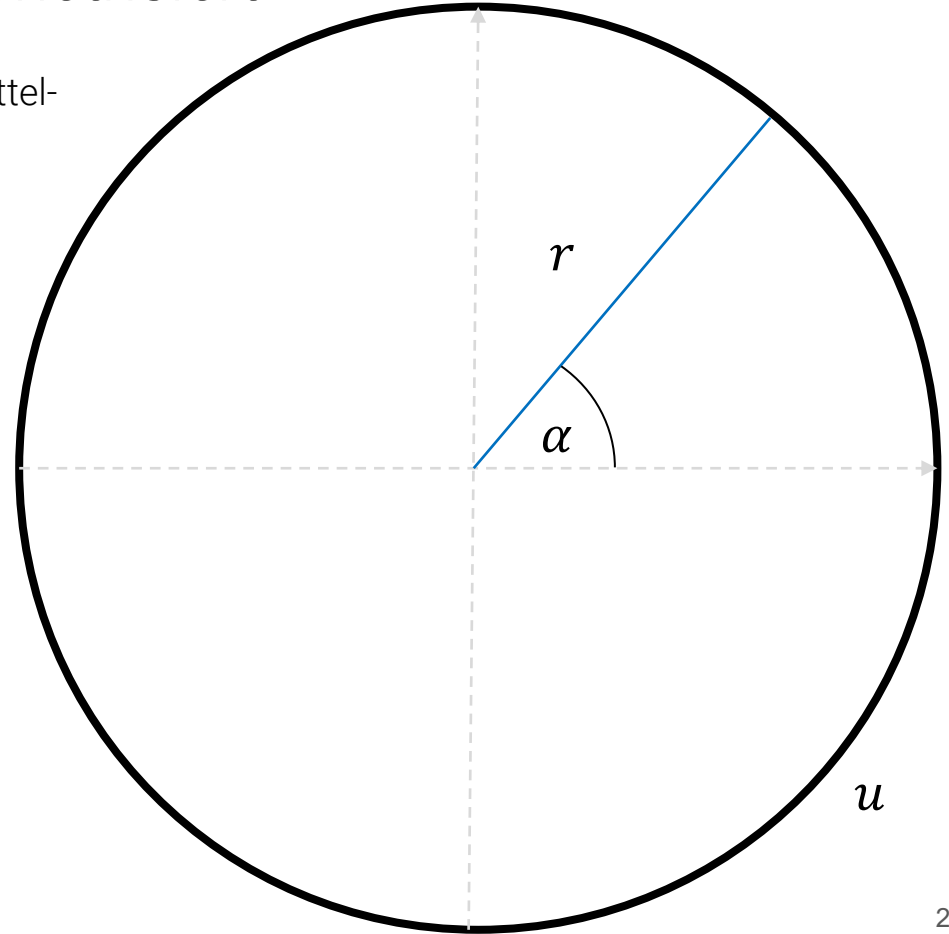
```
x++; if (...) y--; ...
```

Zeichnen eines Kreises – parametrisiert

Der **Kreis** mit dem Radius r , zentriert um den Mittelpunkt $(0,0)$, mit seinen wichtigen Eigenschaften:

`zeichneKreis(x, y, r)`

`zeichneKreis(0, 0, r)`, berechne Kreispunkte
und verschiebe jeden Punkt um den Offset (x, y)





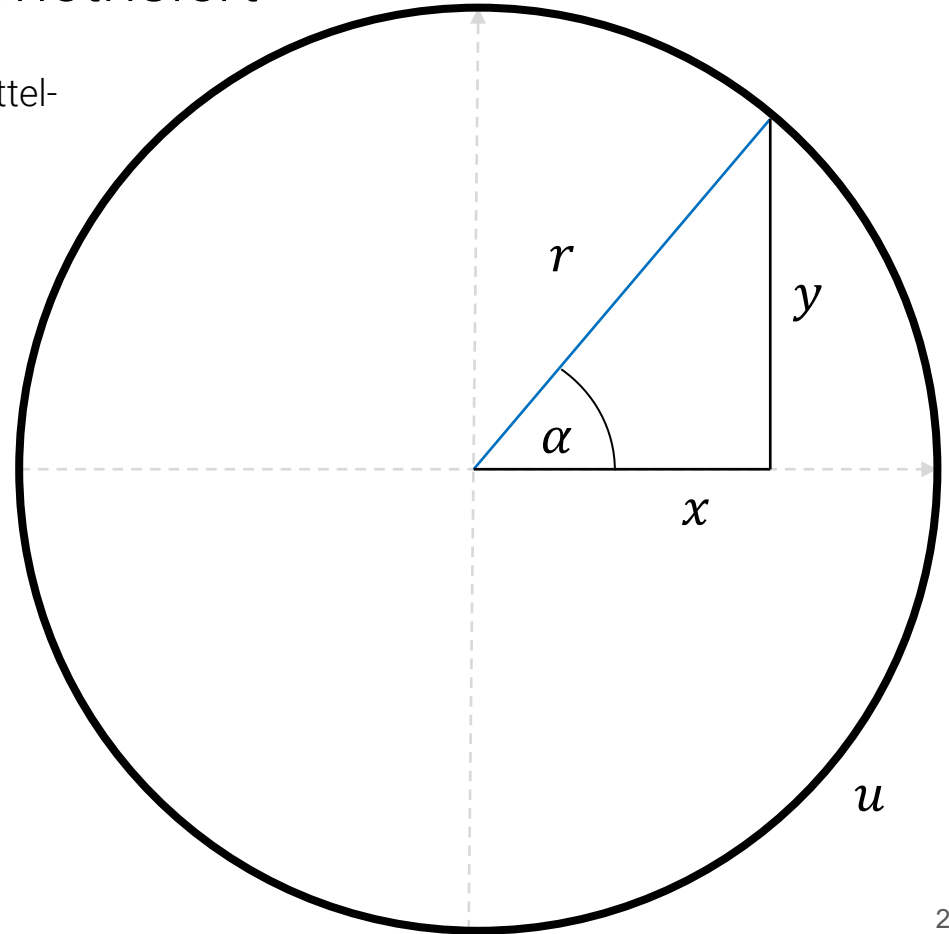
Welchen **Parameter** könnten wir jetzt zum Kreiszeichnen laufen lassen?

Zeichnen eines Kreises – parametrisiert

Der **Kreis** mit dem Radius r , zentriert um den Mittelpunkt $(0,0)$, mit seinen wichtigen Eigenschaften:

$$x = r \cdot \cos(\alpha)$$

$$y = r \cdot \sin(\alpha)$$





Welche **Schrittweite** sollten wir für α wählen?

Zeichnen eines Kreises – parametrisiert

Der Kreis mit dem Radius r , zentriert um den Mittelpunkt $(0,0)$, mit seinen wichtigen Eigenschaften:

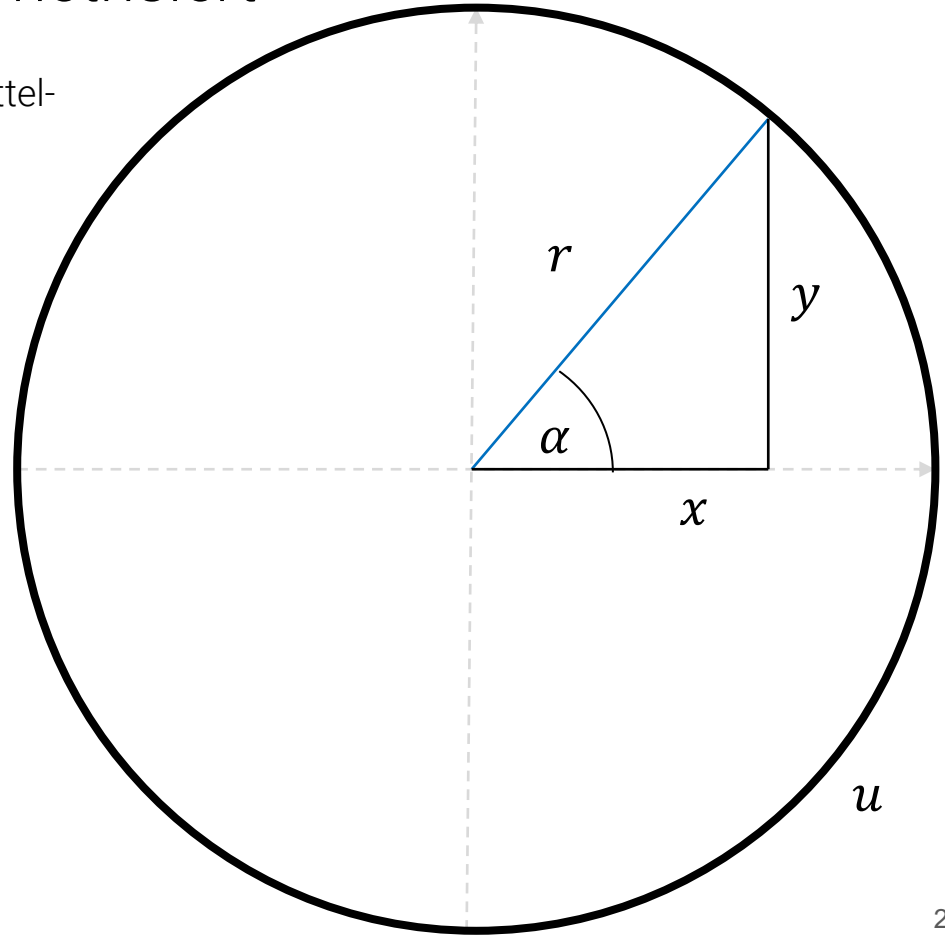
$$x = r \cdot \cos(\alpha)$$

$$y = r \cdot \sin(\alpha)$$

Schrittweite in Bezug auf den Umfang:

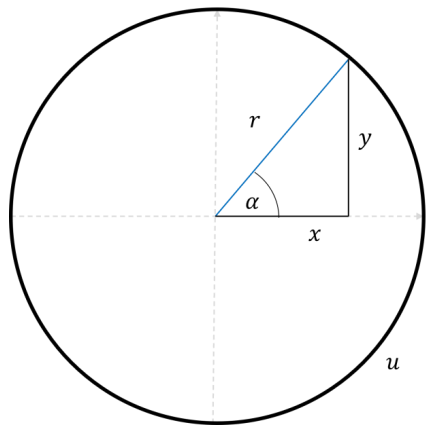
$$u = 2\pi r$$

$$step = \frac{2\pi}{u} = \frac{2\pi}{2\pi r} = \frac{1}{r}$$



Kreis zeichnen – Version 1

Drücken wir die bisherigen Überlegungen mal in einem kleinen Programm zum Kreiszeichnen aus:



$$x = r \cdot \cos(\alpha)$$

$$y = r \cdot \sin(\alpha)$$

$$u = 2\pi r$$

$$step = \frac{2\pi}{u} = \frac{2\pi}{2\pi r} = \frac{1}{r}$$

```
void drawCircleVersion1(int xOff, int yOff, int r) {  
    double step = 1.0/r;  
  
    for (double alpha=0.0; alpha<2*Math.PI; alpha+=step)  
        setPixel(xOff + r*Math.cos(alpha), yOff + r*Math.sin(alpha));  
}
```

Java

Das Programm hat den Nachteil (analog zum Zeichnen von Linien), dass wir **zu viel Gleitkommaarithmetik** verwenden. Wir verwenden diese Gleitkommaarithmetik sowohl für die Berechnung von x als auch für y .



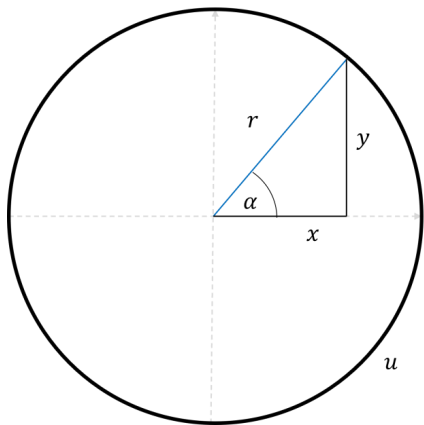
Wie könnten wir den **Berechnungsaufwand** hier wieder verringern?



Beispielsweise könnten wir $\sin$ - und $\cos$ -Werte vorab berechnen und in Tabellen vorbereiten

Kreis zeichnen – Version 2

Die Tabellen für `sin` und `cos` sind für ganzzahlige Winkel bereits vorab berechnet worden. Erweitern wir die bisherigen Überlegungen mal in unserem kleinen Programm zum Kreiszeichnen:



$$x = r \cdot \cos(\alpha)$$

$$y = r \cdot \sin(\alpha)$$

$$u = 2\pi r$$

$$step = \frac{2\pi}{u} = \frac{2\pi}{2\pi r} = \frac{1}{r}$$

```
void drawCircleVersion2(int xOff, int yOff, int r) {  
    double step = 1.0/r;  
  
    for (double alpha=0.0; alpha<2*Math.PI; alpha+=step)  
        setPixel(xOff + r*cos[alpha], yOff + r*sin[alpha]);  
}
```

Java

Die Vorabberechnung von wiederkehrenden Funktionswerten und Bereitstellung von Tabellen, ist ein probates Mittel für die Beschleunigung von performance-relevanten Programmabschnitten.





Warum ist die bisher angewendete Strategie nicht besonders gut geeignet?

Zeichnen eines Kreises – als Funktion

Wir durchlaufen den ersten Quadranten mit ganzzahligem x und drücken y als Funktion von x aus:

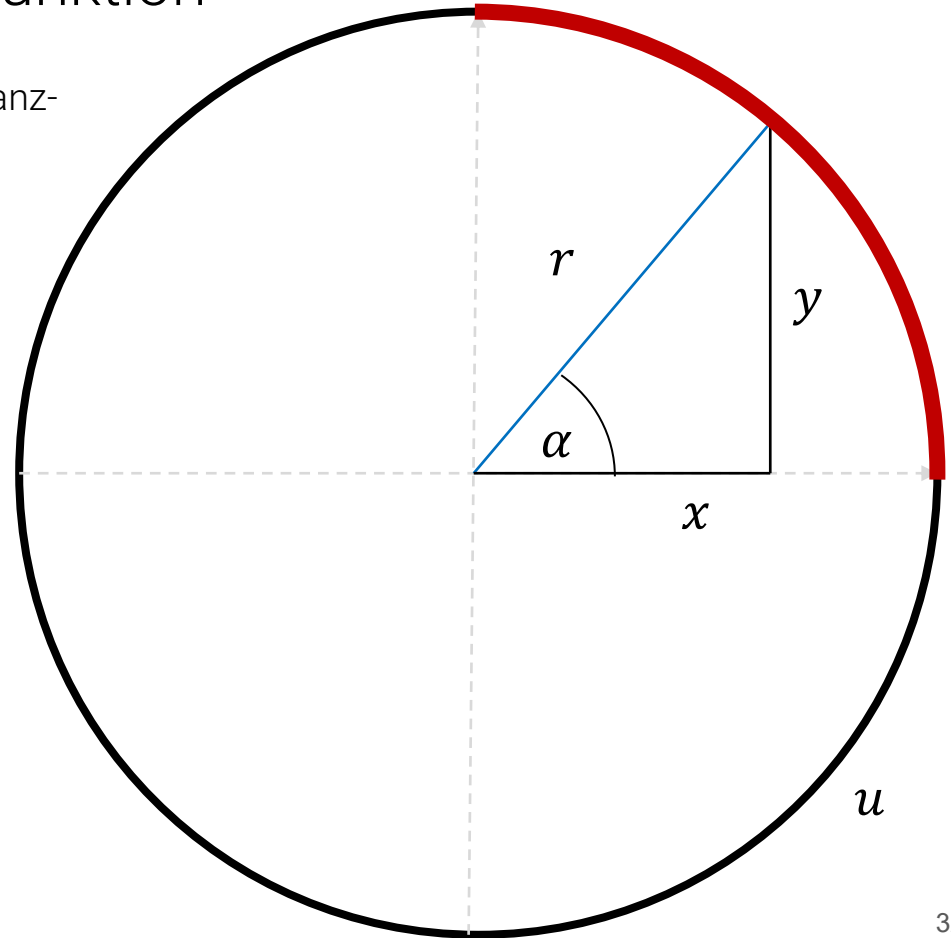
$$x = r \cdot \cos(\alpha)$$

$$y = r \cdot \sin(\alpha)$$

Wir drücken y durch x aus:

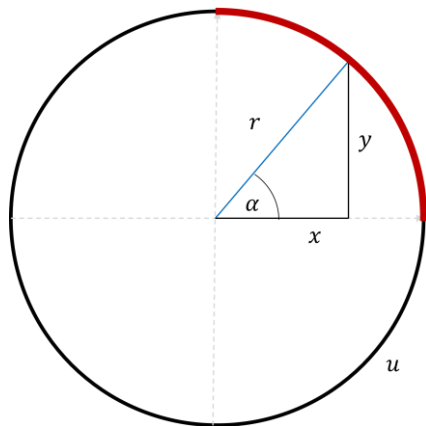
$$x^2 + y^2 = r^2$$

$$y = \sqrt{r^2 - x^2}$$



Kreis zeichnen – Version 3

Wir durchlaufen die x -Werte für den ersten Quadranten und ermitteln den y -Wert über den Satz des Pythagoras:



```
void drawCircleVersion3(int xOff, int yOff, int r) {  
    for (int x=0; x<=r; x++) {  
        double y = Math.sqrt(r*r - x*x);  
        setPixel(xOff + x, yOff + y);  
    }  
}
```

Java

Wir haben mit der neuen Version die Gleitkommaarithmetik um die Hälfte reduziert.

$$x = r \cdot \cos(\alpha)$$

$$y = r \cdot \sin(\alpha)$$

$$x^2 + y^2 = r^2$$

$$y = \sqrt{r^2 - x^2}$$

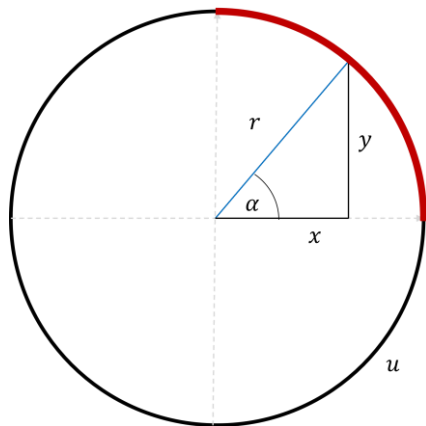




Was ändert sich pro Schritt?
 $-2 \cdot 2, -3 \cdot 3, -4 \cdot 4, \dots$

Kreis zeichnen – Version 3 zu Version 4

Wir durchlaufen die x -Werte für den ersten Quadranten und ermitteln das y über den Satz des Pythagoras:



$$x = r \cdot \cos(\alpha)$$

$$y = r \cdot \sin(\alpha)$$

$$x^2 + y^2 = r^2$$

$$y = \sqrt{r^2 - x^2}$$

```
void drawCircleVersion3(int xOff, int yOff, int r) {  
    for (int x=0; x<=r; x++) {  
        double y = Math.sqrt(r*r - x*x);  
        setPixel(xOff + x, yOff + y);  
    }  
}
```

Java

Schrittweise werden unter der Wurzel die Quadrate erhöht und durchlaufen dabei folgende Werte: $\omega = r^2, r^2 - 1, r^2 - 4, r^2 - 9, \dots$

```
void drawCircleVersion4(int xOff, int yOff, int r) {  
    int d=1, w=r*r;  
    for (int x=0; x<=r; x++) {  
        double y = Math.sqrt(w);  
        setPixel(xOff + x, yOff + y);  
        w=w-d; d=d+2;  
    }  
}
```

ω

r^2

$r^2 - 1$

$r^2 - 4$

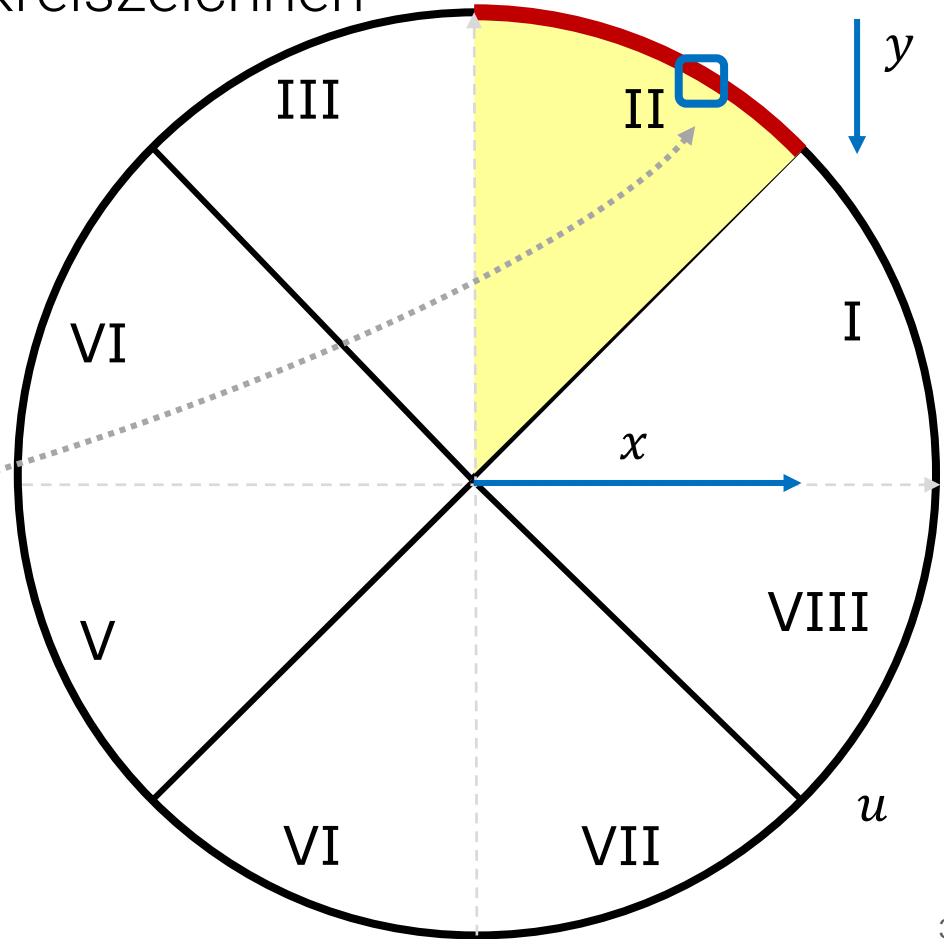
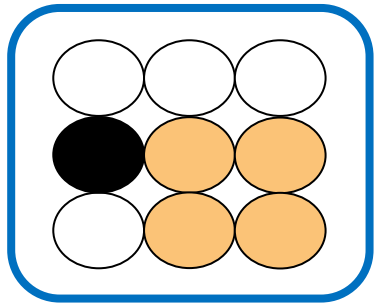
$r^2 - 9$



Jetzt kommen wir zu **Bresenham's Idee**,
Kreise zu zeichnen

Bresenham's Linienidee beim Kreiszeichnen

Wenn wir das Kreiszeichnen zunächst auf den **zweiten Oktanten** beschränken, können wir die Idee von Bresenham, eine Linie zu zeichnen, hier wieder erfolgreich einsetzen:



Idee von Bresenham

Formulieren wir zunächst eine **Entscheidungsfunktion**, die bestimmt, ob ein gegebener Punkt auf dem Kreis, innerhalb oder außerhalb des zu zeichnenden Kreises liegt.

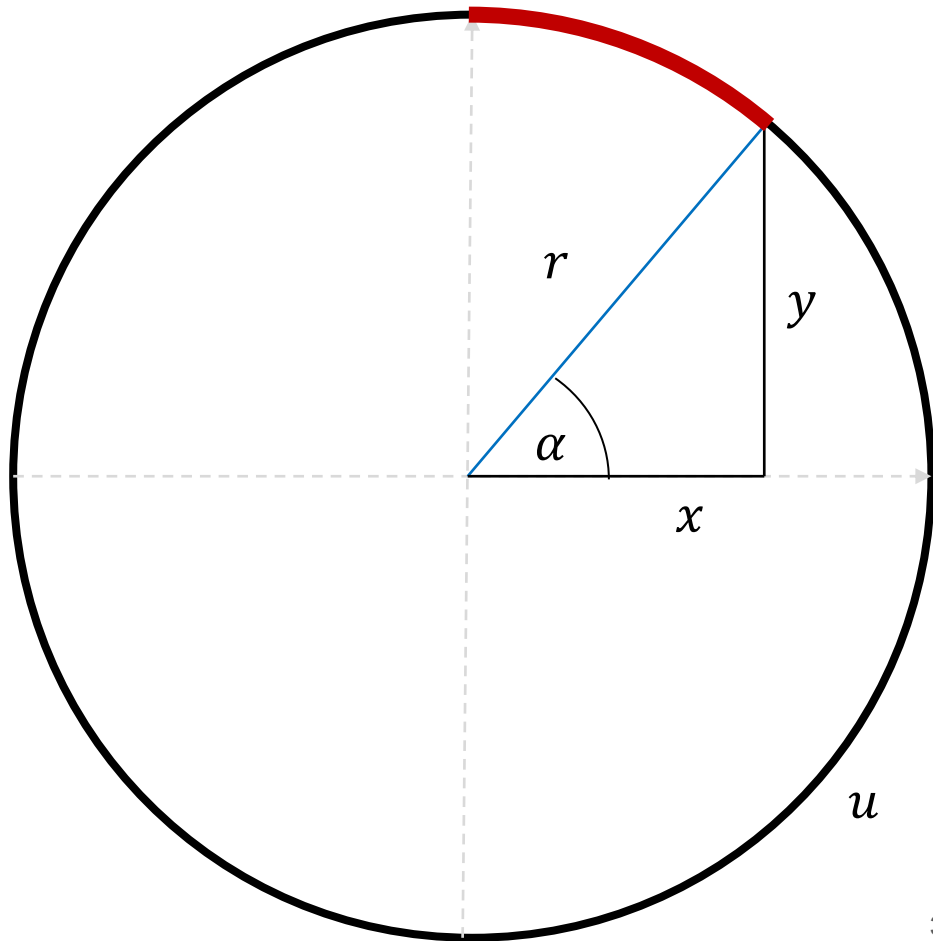
$$x = r \cdot \cos(\alpha)$$

$$y = r \cdot \sin(\alpha)$$

Wir drücken y durch x aus:

$$x^2 + y^2 = r^2$$

$$F(x, y) = x^2 + y^2 - r^2$$





Wie verhält sich die Funktion F bei
einem gegebenen Punkt (x, y) ?

Beobachtung zur Entscheidungsfunktion

Wir wissen, dass $F(x, y) = 0$ ist, für alle Punkte (x, y) , die auf dem Kreis liegen.

Weiterhin gilt:

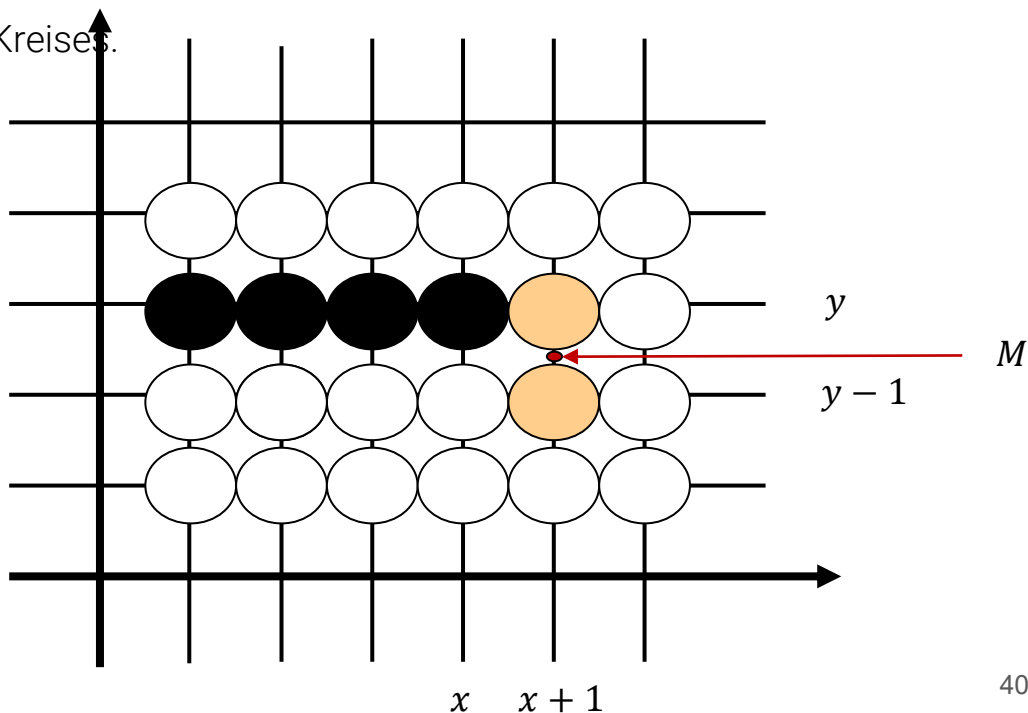
- $F(x, y) < 0$, falls Punkt (x, y) innerhalb des Kreises.
- $F(x, y) \geq 0$, falls Punkt (x, y) außerhalb des Kreises.

Aus dieser Beobachtung heraus konstruieren wir uns eine **Entscheidungsvariable** Δ :

$$\Delta = F(M) = F(x + 1, y - 0.5)$$

$\Delta < 0$, M liegt **innerhalb** des Kreises

$\Delta \geq 0$, M liegt **außerhalb** des Kreises





Wie kann uns Δ bei der Entscheidung helfen?

Entscheidungsvariable nutzen: „außerhalb“

Wir wissen, dass $F(x, y) = 0$ ist, für alle (x, y) , die auf dem Kreis liegen.

Weiterhin gilt:

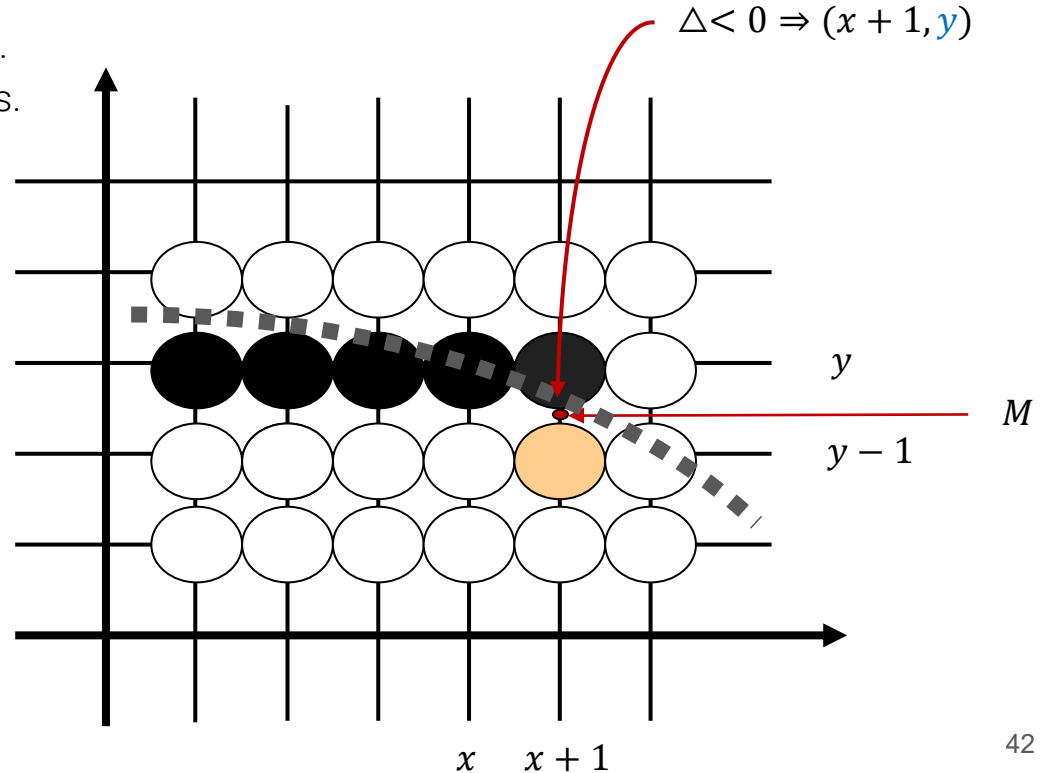
- $F(x, y) < 0$, falls (x, y) innerhalb des Kreises.
- $F(x, y) \geq 0$, falls (x, y) außerhalb des Kreises.

Aus dieser Beobachtung heraus konstruieren wir uns eine Entscheidungsvariable Δ :

$$\Delta = F(M) = F(x + 1, y - 0.5)$$

$\Delta < 0$, M liegt innerhalb des Kreises

$\Delta \geq 0$, M liegt außerhalb des Kreises



Entscheidungsvariable nutzen: „innerhalb“

Wir wissen, dass $F(x, y) = 0$ ist, für alle (x, y) , die auf dem Kreis liegen.

Weiterhin gilt:

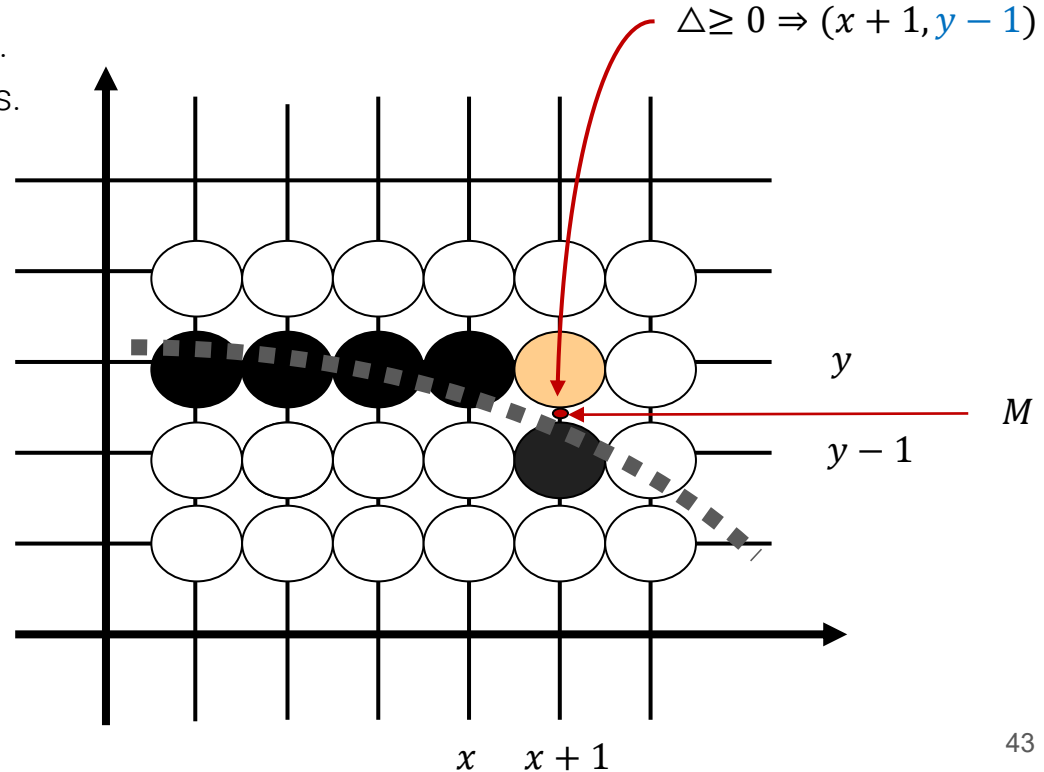
- $F(x, y) < 0$, falls (x, y) innerhalb des Kreises.
- $F(x, y) \geq 0$, falls (x, y) außerhalb des Kreises.

Aus dieser Beobachtung heraus konstruieren wir uns eine Entscheidungsvariable Δ :

$$\Delta = F(M) = F(x + 1, y - 0.5)$$

$\Delta < 0$, M liegt innerhalb des Kreises

$\Delta \geq 0$, M liegt außerhalb des Kreises



Änderungen von $F(x, y)$ unter der Lupe

Die Funktion $F(x, y)$ ist für unsere Verhältnisse noch zu [rechenintensiv](#), daher wollen wir uns die Änderungen nach einer Entscheidung genau anschauen:

$$\Delta = F(M) = F(x + 1, y - 0.5) = (x + 1)^2 + (y - 0.5)^2 - r^2$$

Fall 1: $\Delta < 0$, M liegt innerhalb des Kreises

$$\begin{aligned}\Delta' &= F(x + 2, y - 0.5) \\ &= (x + 2)^2 + (y - 0.5)^2 - r^2 \\ &= (x + 1)^2 + 2x + 3 + (y - 0.5)^2 - r^2 \\ &= \Delta + 2x + 3\end{aligned}$$

Fall 2: $\Delta \geq 0$, M liegt außerhalb des Kreises

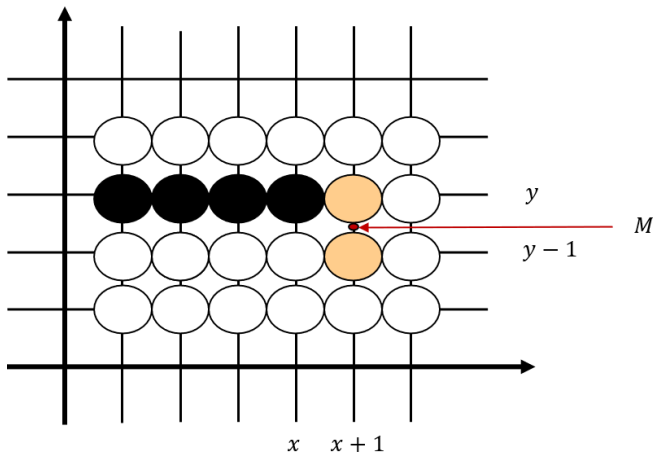
$$\begin{aligned}\Delta' &= F(x + 2, y - 1.5) \\ &= (x + 2)^2 + (y - 1.5)^2 - r^2 \\ &= (x + 1)^2 + 2x + 3 + (y - 0.5)^2 - 2y + 2 - r^2 \\ &= \Delta + 2x - 2y + 5\end{aligned}$$

Das erste Pixel steht bei $(0, r)$. Dann ergibt sich der [Startwert](#) für die Entscheidungsvariable Δ wie folgt:

$$\begin{aligned}\Delta &= F(1, r - 0.5) \\ &= 1^2 + (r - 0.5)^2 - r^2 \\ &= \frac{5}{4} - r\end{aligned}$$

Kreis zeichnen – Bresenhamversion 1

Die gerade verstandene Idee, eine Entscheidungsvariable einzusetzen, wollen wir in einem kleinen Programm zum Ausdruck bringen:



$$\Delta = F(M) = F(x+1, y-0.5)$$

$$\Delta = F(1, r-0.5) = \frac{5}{4} - r$$

$$\Delta' = \Delta + 2x + 3$$

$$\Delta' = \Delta + 2x - 2y + 5$$

```
void drawCircleBresenham1(int xOff, int yOff, int r) {  
    int x=0; int y=r;  
    double delta=5/4.0 - r;  
    while(y>=x) {  
        setPixel(xOff+x, yOff+y);  
        if (delta<0) {  
            delta = delta+2*x+3;  
            x++;  
        } else {  
            delta = delta+2*x-2*y+5;  
            x++; y--;  
        }  
    }  
}
```

Java

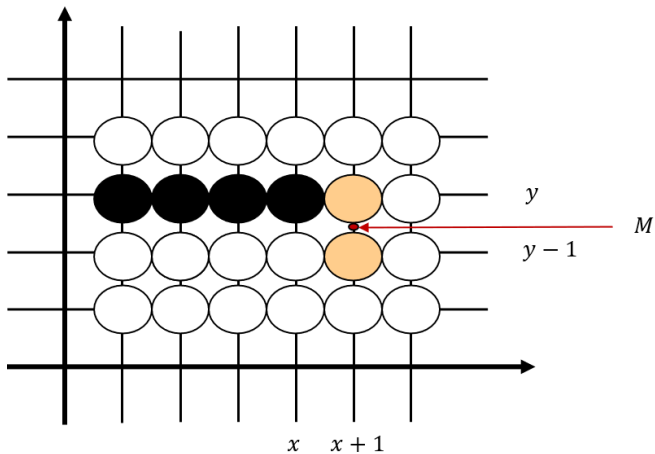
Diese Bresenham-Version ist bereits sehr ausgereift. Aber an welchen Stellen können wir jetzt noch [Gleitkommaarithmetik](#) reduzieren?



An welchen Stellen können wir
Gleitkommaarithmetik ersetzen?

Kreis zeichnen – Bresenhamversion 1

Die gerade verstandene Idee, eine Entscheidungsvariable einzusetzen, wollen wir in einem kleinen Programm zum Ausdruck bringen:



$$\Delta = F(M) = F(x + 1, y - 0.5)$$

$$\Delta = F(1, r - 0.5) = \frac{5}{4} - r$$

$$\Delta' = \Delta + 2x + 3$$

$$\Delta' = \Delta + 2x - 2y + 5$$

```
void drawCircleBresenham1(int xOff, int yOff, int r) {  
    int x=0; int y=r;  
    double delta=5/4.0 - r;  
    while(y>=x) {  
        setPixel(xOff+x, yOff+y);  
        if (delta<0) {  
            delta = delta+2*x+3;  
            x++;  
        } else {  
            delta = delta+2*x-2*y+5;  
            x++; y--;  
        }  
    }  
}
```

Java

$$\Delta \leadsto \Delta - \frac{1}{4}$$

$$\Delta x = 2x + 3$$

$$\Delta xy = 2x - 2y + 5$$

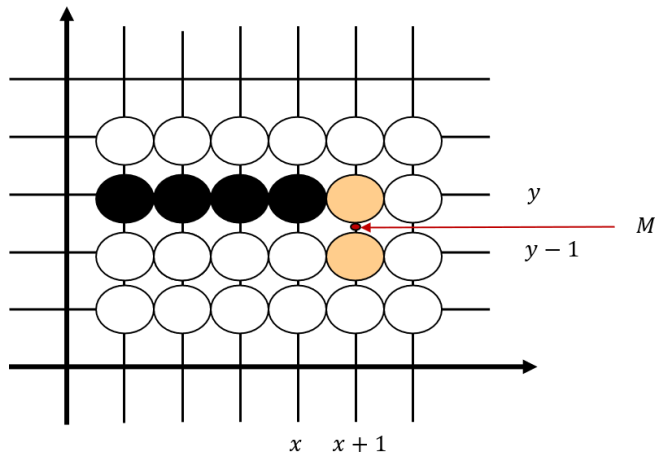
Diese Bresenham-Version ist bereits sehr ausgereift. Aber an welchen Stellen können wir jetzt noch Gleitkommaarithmetik reduzieren?



Schauen wir uns die **optimierte Variante** an.

Kreis zeichnen – Bresenhamversion 2

Die gerade verstandene Idee, eine Entscheidungsvariable einzusetzen, wollen wir in einem kleinen Programm zum Ausdruck bringen:



$$\Delta = F(M) = F(x+1, y-0.5)$$

$$\Delta = F(1, r-0.5) = \frac{5}{4} - r$$

$$\Delta' = \Delta + 2x + 3$$

$$\Delta' = \Delta + 2x - 2y + 5$$

```
void drawCircleBresenham2(int xOff, int yOff, int r) {  
    int x=0; int y=r; int delta = 1-r;  
    int deltaX = 3;    int deltaXY = -2*r+5;  
    while(y>=x) {  
        setPixel(xOff+x, yOff+y);  
        if (delta<0) {  
            delta = delta+deltaX;  
            deltaX +=2; deltaXY+=2;  
            x++;  
        } else {  
            delta = delta+deltaXY;  
            deltaX +=2; deltaXY+=4;  
            x++; y--;  
        }  
    }  
}
```

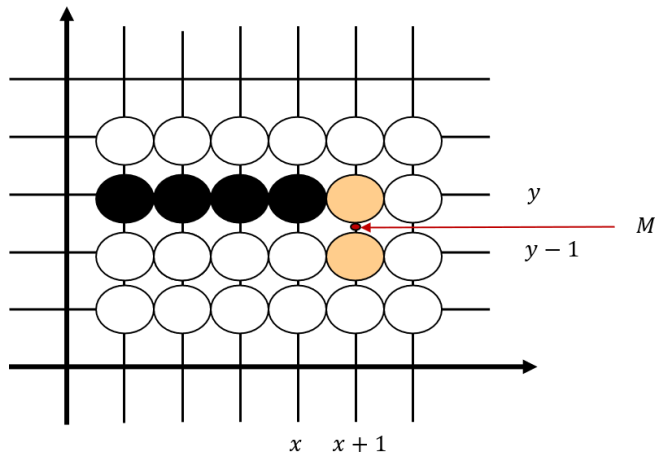
Java



Aber warum genügt es, den Startwert $\Delta \leadsto \Delta - 1/4$ zu ändern, ohne die Änderung für **if** ($\Delta < -1/4$) als Entscheidung zu übernehmen?

Kreis zeichnen – Bresenhamversion 2

Die gerade verstandene Idee, eine Entscheidungsvariable einzusetzen, wollen wir in einem kleinen Programm zum Ausdruck bringen:



$$\Delta = F(M) = F(x+1, y-0.5)$$

$$\Delta = F(1, r-0.5) = \frac{5}{4} - r$$

$$\Delta' = \Delta + 2x + 3$$

$$\Delta' = \Delta + 2x - 2y + 5$$

```
void drawCircleBresenham2(int xOff, int yOff, int r) {
    int x=0; int y=r; int delta = 1-r;  $\Delta \leadsto \Delta - \frac{1}{4}$ 
    int deltaX = 3;    int deltaXY = -2*r+5;
    while(y>=x) {
        setPixel(xOff+x, yOff+y);
        if (delta<0) {
            delta = delta+deltaX;
            deltaX +=2; deltaXY+=2;
            x++;
        } else {
            delta = delta+deltaXY;
            deltaX +=2; deltaXY+=4;
            x++; y--;
        }
    }
}
```

Java

$$\Delta \leadsto \Delta - \frac{1}{4}$$

if (delta< -1/4.0)

Da das neue **delta** zunächst mit ganzzahligen Werten initialisiert wird (**1-r**) und später nur ganzzahlig erhöht wird (in beiden Fällen), können wir **-1/4.0** durch **0** ersetzen.

Idee von Bresenham

Formulieren wir zunächst eine **Entscheidungsfunktion**, die bestimmt, ob ein gegebener Punkt auf dem Kreis, innerhalb oder außerhalb des zu zeichnenden Kreises liegt.

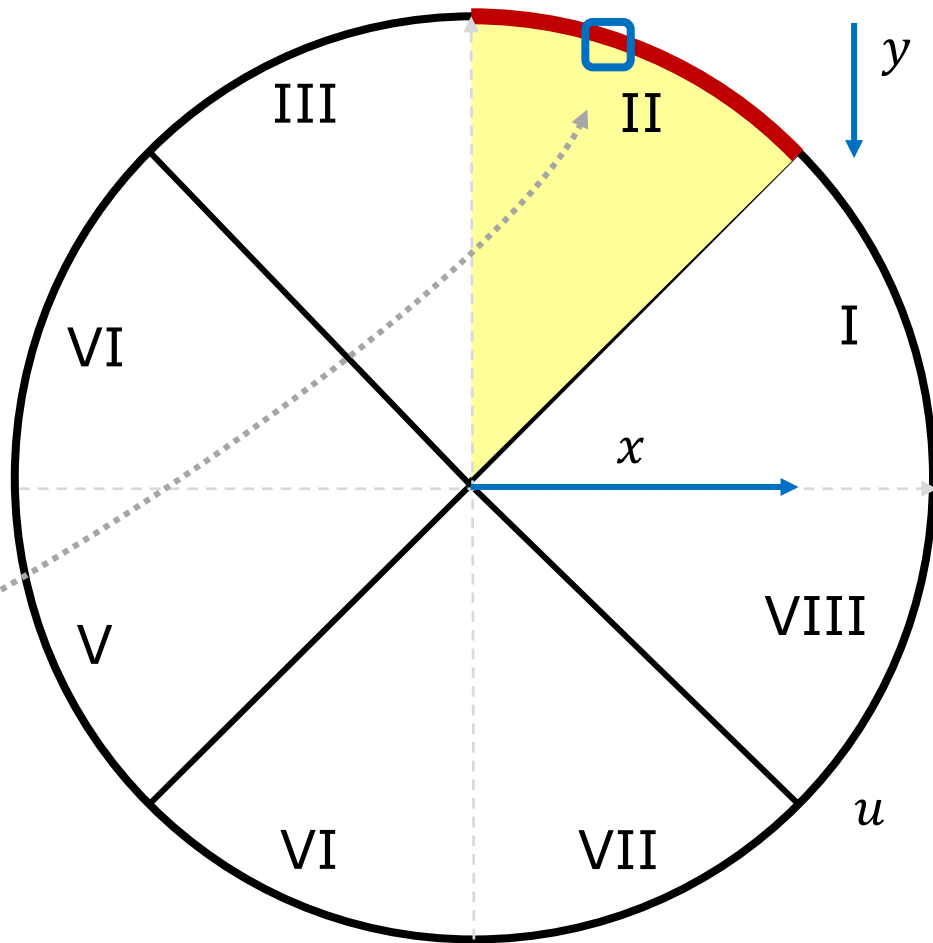
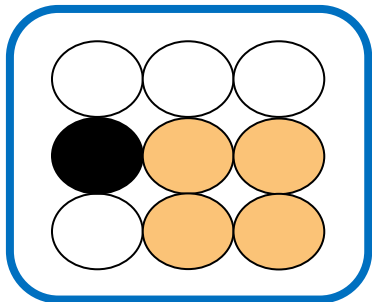
$$x = r \cdot \cos(\alpha)$$

$$y = r \cdot \sin(\alpha)$$

Wir drücken y durch x aus:

$$x^2 + y^2 = r^2$$

$$F(x, y) = x^2 + y^2 - r^2$$

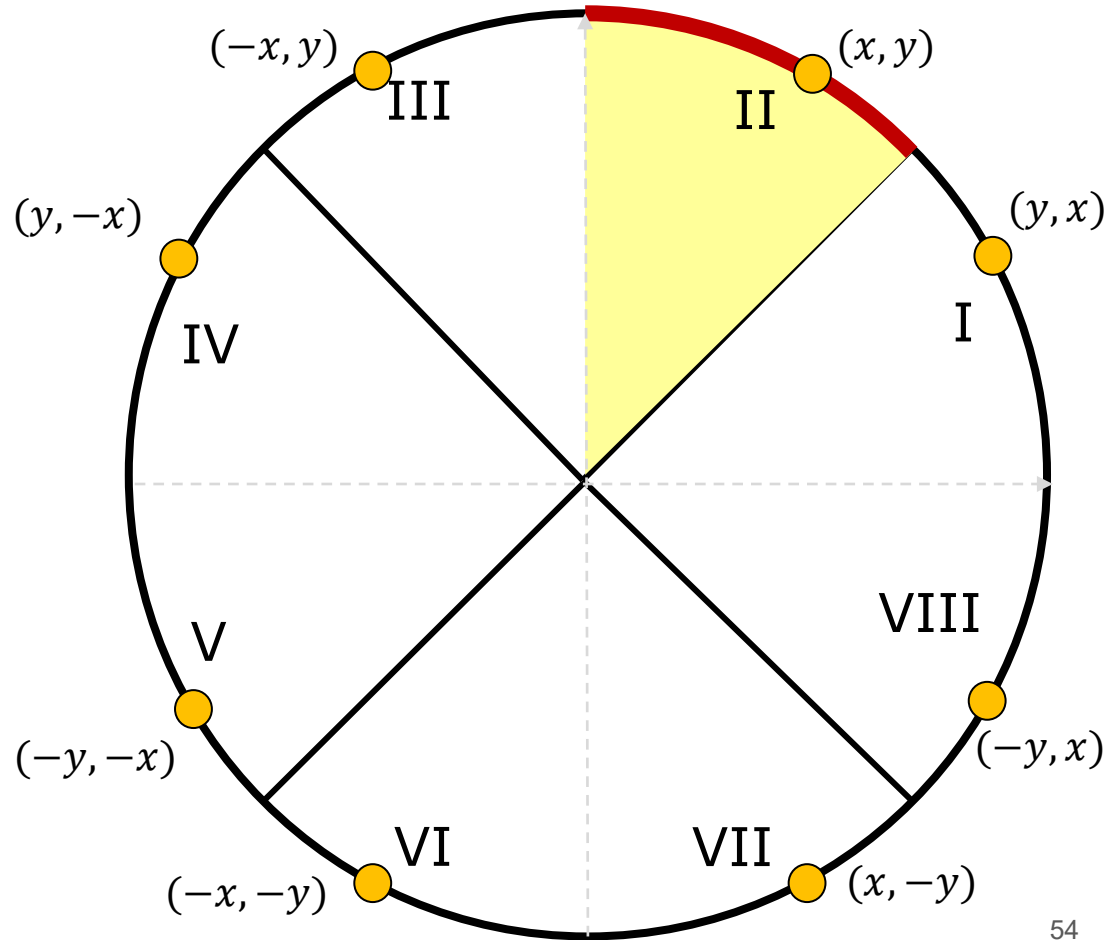




Was machen wir jetzt, um den
vollständigen Kreis zu zeichnen?

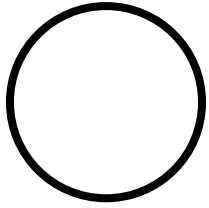
Kreis vollständig zeichnen

Wir können aufgrund der Symmetrie, alle Oktanten gleichzeitig zeichnen.



Kreisalgorithmen im Überblick

Hier sehen wir noch einmal die drei vorgestellten Kreisalgorithmen in der Übersicht. Zu erkennen sind hier in den Beispielen deutliche Unterschiede gerade bei den Kreisen mit geringeren Radien.



Parametrisiert:

$$x = f_x(t) \quad y = f_y(t)$$

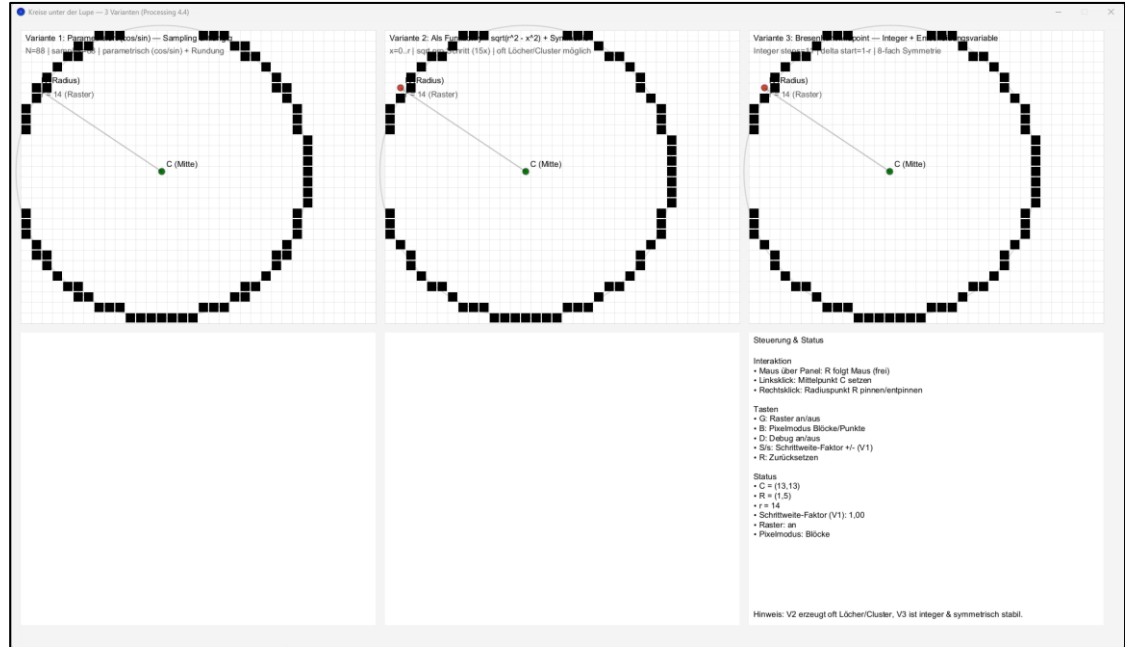
Funktionsgleichung (Kreis):

$$y = f(x)$$

Bresenham-Kreis:

```
x++; if (...) y--; ...
```

polygone/kreisalgorithmen





Jetzt zeichnen wir **Ellipsen**.

Kreis zu Ellipse – parametrisiert

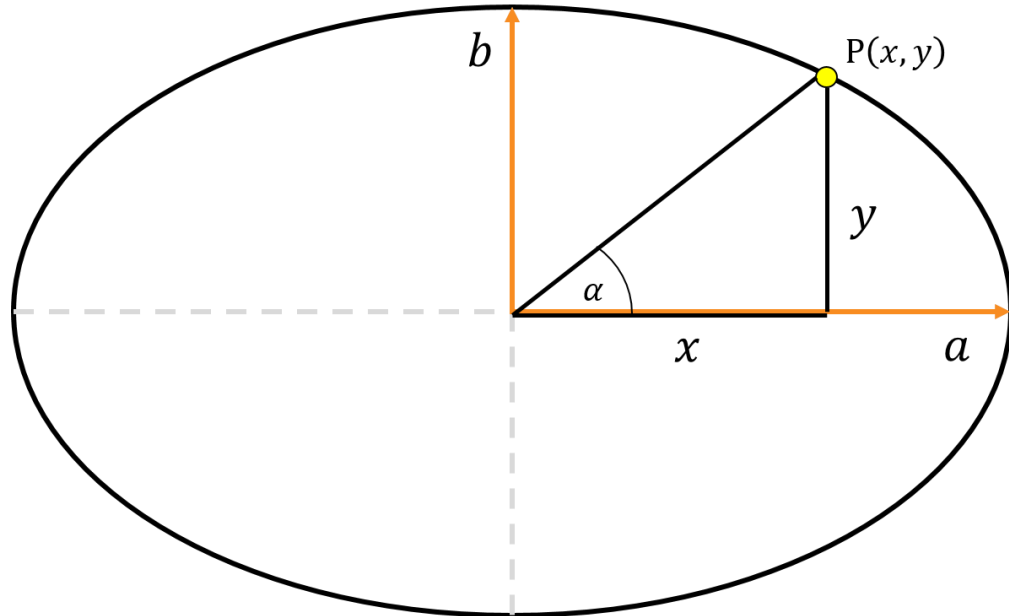
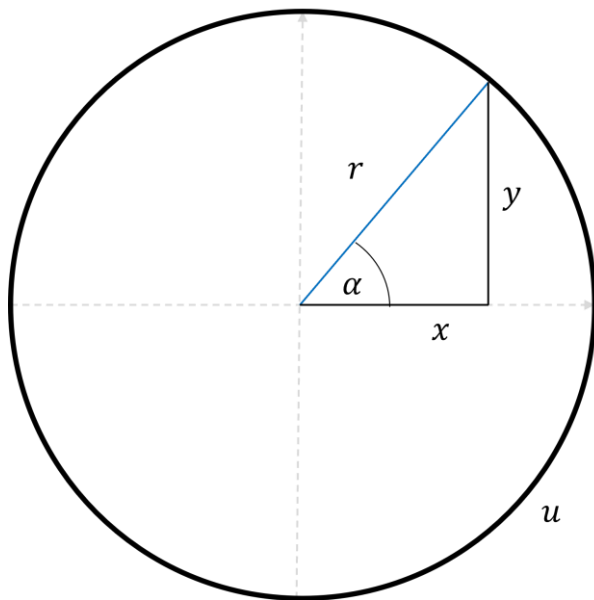
Der Zusammenhang zwischen Kreis und Ellipse ist offensichtlich [1]:

$$x = r \cdot \cos(\alpha)$$

$$y = r \cdot \sin(\alpha)$$

$$x = a \cdot \cos(\alpha)$$

$$y = b \cdot \sin(\alpha)$$

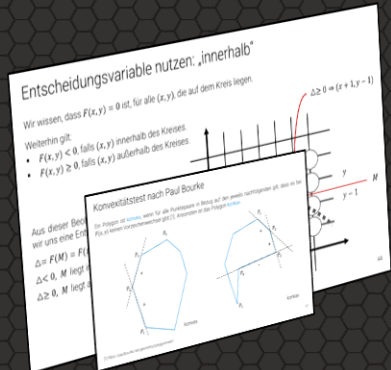


Ein Punkt $P(x, y)$ gehört zu einer Ellipse (a, b) , wenn gilt

$$\frac{x^2}{a^2} + \frac{y^2}{b^2} = 1$$

Diskrete Geometrie: Konvexität und Kreisalgorithmen

Key Takeaways



- Unterschied zwischen konvexen und konkaven Polygonen
- Verwendung orientierter Flächen/Vorzeichenfunktionen
- Performance-Probleme durch Floating-Point-Arithmetik
- Inkrementelle Algorithmen zur Rasterisierung
- Bresenham's Grundidee für Kreise
- Nutzung von Symmetrien zur Effizienzsteigerung
- Übertragung vom Kreis zur Ellipse

Fragestellungen zum Vorlesungsteil

Im Anschluss an diese Veranstaltung sollten Sie die folgenden Fragen sicher beantworten können:

- (1) Beschreiben Sie den Unterschied zwischen konkaven und konvexen Polygonen.
- (2) Erläutern Sie den Algorithmus von Paul Bourke zur Idee des Konvexitätstests für Polygone.
- (3) Überlegen Sie sich, wann der Algorithmus von Paul Bourke für ein Polygon mit n Punkten besonders lange arbeitet und in welchem Fall er sehr schnell eine Antwort finden kann. Können Sie die Laufzeit im Best- und Worst-Case bestimmen?
- (4) Welchen visuellen Nachteil hat das vorgestellte Verfahren zum Kreiszeichnen mit der Funktionsdarstellung, wenn wir von $x=0$ bis r schrittweise Pixel zeichnen wollen?
- (5) Bresenham nutzt beim Zeichnen eines Kreises zusätzlich die Symmetrieeigenschaft aus. Erläutern Sie konkret für alle auftretenden Fälle, wie er das macht und geben Sie die resultierenden Punkte an.
- (6) Geben Sie ein Beispielpolygon an, bei dem der Bourke-Test bei Selbstüberschneidung eine irreführende Aussage liefert.



Praktische Aufgaben

Folgende praktische Aufgaben sollten Sie jetzt selbstständig durchführen:

Aufgabe 1) [Processing] Implementieren Sie den Konvexitätstest (nach Paul Bourke) einfacher Polygone.

Aufgabe 2) [Processing] Implementieren Sie drei Varianten zum Zeichnen von Ellipsen (parametrisiert, als Funktion und über Bresenham).





Vielen Dank für die Aufmerksamkeit